

Laporan Tugas Kecil 3

IF2211 STRATEGI ALGORITMA

ICE SLIDING PUZZLE SOLVER

SEMESTER II / 2025–2026



Disusun oleh:

Nicholas Wise Saragih Sumbayak (13524037)

Reinhard Alfonzo Hutabarat (13524056)

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

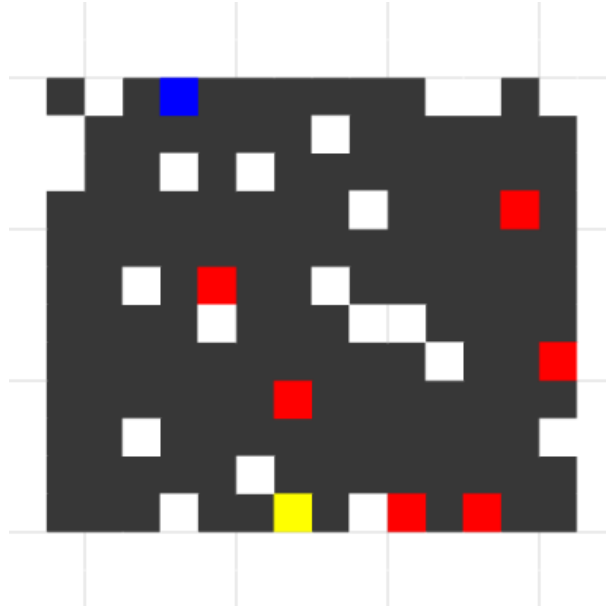
Daftar Isi

1	Pendahuluan	3
2	Dasar Teori	3
2.1	Representasi Permasalahan sebagai Graf	3
2.2	Uniform Cost Search (UCS)	4
2.2.1	Langkah-Langkah Algoritma UCS	4
2.3	Greedy Best-First Search (GBFS)	5
2.3.1	Langkah-Langkah Algoritma GBFS	5
2.4	A* Search (A-Star)	6
2.4.1	Langkah-Langkah Algoritma A*	6
2.5	Iterative Deepening A* (IDA*)	7
2.5.1	Langkah-Langkah Algoritma IDA*	7
2.6	Admissibilitas Heuristik	7
2.7	Perbandingan Teoritis Algoritma	8
3	Desain dan Implementasi	9
3.1	Simulasi Gerakan Sliding	10
3.2	Desain Heuristik	10
3.3	Arsitektur Program	11
3.4	Struktur Proyek	12
4	Eksperimen dan Hasil Pengujian	13
4.1	Konfigurasi Test Case	13
4.2	UCS (Uniform Cost Search)	14
4.2.1	UCS dengan Input 1	14
4.3	GBFS (Greedy Best-First Search)	14
4.3.1	GBFS Heuristik 1 dengan Input 2	14
4.3.2	GBFS Heuristik 2 dengan Input 2	15
4.3.3	GBFS Heuristik 3 dengan Input 2	16
4.4	A* Search	16
4.4.1	A* Heuristik 1 dengan Input 3	16
4.4.2	A* Heuristik 2 dengan Input 3	17
4.4.3	A* Heuristik 3 dengan Input 3	18
4.5	IDA* (Iterative Deepening A*)	18
4.5.1	IDA* Heuristik 1 dengan Input 4	18
4.5.2	IDA* Heuristik 2 dengan Input 4	19
4.5.3	IDA* Heuristik 3 dengan Input 4	20
4.6	Analisis Keseluruhan	20
4.6.1	Perbandingan Performa Algoritma	20
4.6.2	UCS — Kelebihan dan Keterbatasan	21
4.6.3	GBFS — Kelebihan dan Keterbatasan	21
4.6.4	A* — Kelebihan dan Keterbatasan	21
4.6.5	IDA* — Kelebihan dan Keterbatasan	22
5	Kesimpulan dan Saran	22
5.1	Kesimpulan	22
5.2	Saran Pengembangan	22

5.3	Refleksi Individu	23
5.3.1	Nicholas Wise Saragih Sumbayak (13524037)	23
5.3.2	Reinhard Alfonzo Hutabarat (13524056)	23
Referensi		24
Lampiran		25
Pernyataan Keaslian		25

1 Pendahuluan

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Mekanik utama dari permainan ini adalah karakter (pin) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan yang licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan batu.



Gambar 1: Permainan *Ice Sliding Puzzle*

Tujuan akhir dari Tugas Kecil ini adalah menemukan jalur solusi yang memandu karakter utama menuju titik tujuan secara selamat. Dalam perjalanannya, pemain harus menghindari wilayah lava (kotak merah) yang akan menyebabkan *game over* jika dilewati. Selain itu, terdapat modifikasi *constraint* tambahan berupa *tile* berangka; pemain diwajibkan untuk melewati *tile* angka tersebut sesuai dengan urutan membesar secara berurutan.

2 Dasar Teori

2.1 Representasi Permasalahan sebagai Graf

Permasalahan *Ice Sliding Puzzle* dapat dimodelkan sebagai masalah pencarian jalur (*pathfinding*) di dalam sebuah graf ruang keadaan (*state space graph*). Dalam konteks ini, graf tidak direpresentasikan secara statis di awal, melainkan dibangun secara dinamis seiring dengan berjalannya eksplorasi agen. Simpul (*node*) dalam graf ini merepresentasikan status atau keadaan (*state*) spesifik dari permainan pada suatu waktu. Keadaan ini tidak sekadar bergantung pada letak koordinat spasial baris dan kolom agen, melainkan secara esensial mencakup parameter kemajuan *checkpoint* berupa urutan angka yang telah diambil. Sisi (*edge*) pada graf melambangkan transisi antar keadaan yang valid, yaitu aksi meluncur (*sliding*) ke salah satu dari empat arah kardinal hingga agen berhenti akibat menabrak tembok atau rintangan. Biaya dari setiap sisi didefinisikan sebagai akumulasi biaya (*cost*) dari seluruh petak yang dilalui agen selama proses meluncur dalam

satu tindakan.

2.2 Uniform Cost Search (UCS)

Algoritma *Uniform Cost Search* (UCS) merupakan algoritma pencarian tidak berinformasi (*uninformed search*) yang selalu mengekskansi simpul dengan biaya lintasan kumulatif terkecil dari simpul awal. Algoritma ini menggunakan struktur data antrean prioritas (*priority queue*) untuk mengurutkan daftar simpul berdasarkan nilai fungsi objektifnya. Pada simulasi *Ice Sliding Puzzle*, UCS secara sekuensial akan mengekskansi arah lintasan agen, memvalidasi pergerakan (misalnya menghentikan dan menggugurkan transisi jika agen menyentuh petak lava mematikan), lalu mengakumulasi biaya pergerakan petak demi petak untuk diuji tingkat prioritasnya.

Secara formal dalam konteks teka-teki ini, fungsi evaluasi murni dikendalikan oleh $g(n)$. Definisi dari $g(n)$ adalah total biaya traversal aktual yang dibutuhkan untuk mencapai keadaan n dari posisi pijakan paling awal. Biaya akumulatif ini dikalkulasi dengan menjumlahkan *cost* spesifik pada setiap ubin pijakan yang telah dilewati agen, secara eksklusif mengecualikan petak asal permulaan tolakan, namun tetap mencakup ubin dinding tempat pendaratan akhirnya berhenti bergeser. Karena algoritma terus-menerus mencari lintasan kumulatif termurah serta menyimpan riwayat kunci kombinasi posisi dan target angka (*state key*) di dalam *visited map* anti-siklus, UCS terjamin menemukan rute dengan biaya paling minim selama biaya setiap sisi bernilai non negatif [1].

2.2.1 Langkah-Langkah Algoritma UCS

1. **Inisialisasi:** Membangun sebuah antrean prioritas (*priority queue*) dan memasukkan *initial state* yang direpresentasikan oleh `models.GameState` berisi posisi awal aktor (Z) dan nilai `NextNum` yang disesuaikan dengan konfigurasi papan: bernilai 0 apabila terdapat angka pada papan, atau -1 apabila tidak ada angka sama sekali. Biaya kumulatif awal $g = 0$ ditetapkan untuk *initial state* tersebut.
2. **Struktur Perulangan:** Melakukan iterasi selama antrean prioritas tidak dalam keadaan kosong.
3. **Ekstraksi Node:** Mengambil simpul dengan total biaya aktual ($g(n)$) terkecil dari antrean prioritas untuk dievaluasi.
4. **Pengecekan Visited Set:** Memeriksa apakah kombinasi posisi koordinat dan progres *checkpoint* (`NextNum`) pada simpul tersebut sudah terdapat dalam *visited set*. Jika sudah pernah dikunjungi dengan biaya yang lebih rendah, maka simpul diabaikan untuk menghindari siklus tak terbatas.
5. **Uji Tujuan (*Goal Test*):** Memeriksa apakah aktor telah mencapai koordinat titik keluar (0) dan variabel `NextNum` telah bernilai -1 (menandakan seluruh urutan angka 0–9 telah diambil). Jika terpenuhi, algoritma berhenti dan mengembalikan jalur solusi.
6. **Ekskansi dan Simulasi Meluncur:** Melakukan simulasi aksi *sliding* ke empat arah kardinal. Untuk setiap arah, aktor bergerak hingga menabrak dinding (X) atau batas papan. Selama meluncur, biaya setiap ubin yang dilewati diakumulasikan ke dalam biaya gerak (*edge cost*), kecuali ubin posisi awal gerakan tersebut.

7. **Validasi State:** Jika dalam lintasan meluncur aktor melewati petak lava (L) atau keluar dari batas papan tanpa menabrak rintangan, maka aksi tersebut dinyatakan gugur (*invalid*). Jika valid, simpul baru dengan biaya kumulatif yang telah diperbarui dimasukkan ke dalam antrean prioritas.
8. **Terminasi:** Jika antrean kosong sebelum tujuan tercapai, algoritma berhenti dan menyatakan bahwa solusi tidak ditemukan.

2.3 Greedy Best-First Search (GBFS)

Berbeda dengan UCS, *Greedy Best-First Search* (GBFS) merupakan algoritma pencarian berinformasi (*informed search*) yang menggunakan fungsi evaluasi:

$$f(n) = h(n)$$

di mana $h(n)$ adalah estimasi biaya dari simpul n menuju tujuan (*goal*). GBFS sama sekali tidak mempertimbangkan biaya perjalanan yang telah ditempuh ($g(n)$ diabaikan), sehingga eksplorasi sepenuhnya dipandu oleh intuisi heuristik menuju sasaran [1].

Dalam konteks *Ice Sliding Puzzle*, GBFS pada setiap iterasi memilih arah luncuran yang menghasilkan posisi berhenti dengan nilai $h(n)$ terkecil, yaitu posisi yang diperkirakan paling dekat dengan *checkpoint* berikutnya atau titik tujuan. Pendekatan ini menghasilkan kecepatan eksekusi yang tinggi karena algoritma cenderung langsung bergerak menuju sasaran. Namun, karena biaya aktual perjalanan diabaikan, GBFS tidak menjamin solusi yang optimal dan berisiko menghasilkan jalur dengan total biaya yang lebih tinggi dibanding solusi terbaik [1].

2.3.1 Langkah-Langkah Algoritma GBFS

1. **Inisialisasi:** Memasukkan *initial state* ke dalam antrean prioritas berdasarkan nilai fungsi heuristik $h(n)$ yang menghitung estimasi jarak sisa menuju tujuan atau *checkpoint* berikutnya.
2. **Ekstraksi Berbasis Heuristik:** Dalam setiap iterasi, algoritma mengambil simpul yang memiliki nilai estimasi heuristik terkecil, tanpa mempertimbangkan biaya historis yang telah ditempuh.
3. **Pengecekan Duplikasi:** Menggunakan *visited set* dengan kunci kombinasi posisi dan progres angka (*NextNum*) untuk memastikan algoritma tidak terjebak dalam eksplorasi berulang pada keadaan yang sama.
4. **Prosedur Meluncur:** Mensimulasikan gerakan meluncur lurus hingga aktor berhenti oleh rintangan dinding. Setiap pergerakan divalidasi terhadap keberadaan petak lava; jika aktor menyentuh lava di tengah lintasan, simpul anak tersebut tidak akan dimasukkan ke dalam daftar eksplorasi.
5. **Pembaruan Progres Checkpoint:** Jika selama gerakan meluncur aktor melewati angka yang sesuai dengan urutan target (*NextNum*), maka status *NextNum* pada simpul baru diperbarui ke angka berikutnya hingga mencapai nilai -1 setelah angka terakhir diambil.
6. **Terminasi:** Algoritma berakhir dengan sukses jika *goal test* terpenuhi, atau berakhir dengan kegagalan jika seluruh ruang keadaan yang dapat dijangkau telah dieksplorasi tanpa menemukan titik tujuan.

2.4 A* Search (A-Star)

Algoritma A* (*A-Star*) menggabungkan keunggulan UCS dan GBFS dengan mendefinisikan fungsi evaluasi [2]:

$$f(n) = g(n) + h(n)$$

Algoritma ini mempertimbangkan biaya aktual perjalanan sekaligus estimasi sisa biaya menuju tujuan. Dalam konteks *Ice Sliding Puzzle*, ketiga komponen fungsi evaluasi didefinisikan sebagai berikut:

- $f(n)$: estimasi total biaya solusi dari posisi awal menuju tujuan apabila jalur tersebut melewati simpul n . Simpul dengan $f(n)$ terkecil diprioritaskan untuk diekspansi.
- $g(n)$: biaya aktual kumulatif yang telah dikeluarkan untuk mencapai simpul n dari posisi awal aktor (Z). Nilai ini dihitung dengan menjumlahkan biaya traversal seluruh petak yang telah dilalui sepanjang jalur dari awal hingga simpul n , dengan ketentuan bahwa petak posisi awal setiap gerakan tidak dihitung, namun petak posisi berhenti tetap dihitung.
- $h(n)$: estimasi biaya minimum dari simpul n menuju tujuan, dihitung menggunakan fungsi heuristik. Nilai ini harus bersifat *admissible*, yaitu tidak pernah melebihi biaya aktual yang sesungguhnya ($h(n) \leq h^*(n)$).

Dengan mempertahankan *visited set* dan menggunakan $f(n)$ sebagai prioritas eksplorasi, A* terjamin menemukan solusi optimal selama heuristik yang digunakan bersifat *admissible* [2].

2.4.1 Langkah-Langkah Algoritma A*

1. **Inisialisasi:** Mempersiapkan antrean prioritas dan memasukkan simpul awal dengan nilai prioritas $f(n) = g(n) + h(n)$. Nilai $g(n)$ adalah biaya aktual (0) dan $h(n)$ adalah estimasi jarak manhattan menuju rangkaian *checkpoint*.
2. **Ekstraksi Optimal:** Mengambil simpul dengan total nilai estimasi biaya terendah ($f(n)$) untuk diekspansi. Penggunaan $g(n)$ memastikan pencarian tetap memperhatikan biaya traversal ubin, sementara $h(n)$ memberikan arahan menuju tujuan.
3. **Manajemen Visited:** Mencatat setiap keadaan (*posisi*, *NextNum*) yang telah dievaluasi. Jika simpul yang sama ditemukan kembali dengan biaya $g(n)$ yang lebih besar, simpul tersebut akan diabaikan untuk menjamin efisiensi.
4. **Mekanisme Gerakan:** Melakukan *sliding* ke empat arah. Algoritma menghitung jumlah biaya (*cost*) dari seluruh ubin yang dilintasi dari titik setelah keberangkatan hingga titik berhenti di depan dinding rintangan.
5. **Validasi Keamanan:** Memastikan aktor tidak melewati petak lava (L) dan tidak meluncur keluar dari batas arena permainan. Gerakan yang melanggar ketentuan ini dianggap tidak valid dan simpul anaknya tidak akan dibangkitkan.
6. **Goal Test dan Solusi:** Pencarian dinyatakan berhasil jika simpul yang diekstraksi berada di lokasi tujuan (0) dengan status seluruh angka (*checkpoint*) telah terambil sesuai urutan.

2.5 Iterative Deepening A* (IDA*)

*Iterative Deepening A** (IDA*) merupakan pengembangan dari algoritma A* yang dirancang untuk mengatasi keterbatasan penggunaan memori. Algoritma A* konvensional menyimpan seluruh simpul pada *frontier* di dalam antrean prioritas, sehingga konsumsi memori dapat tumbuh secara eksponensial pada papan yang besar. IDA* mengatasi hal ini dengan menjalankan *Depth-First Search* (DFS) secara berulang (*iterative*), dibatasi oleh nilai ambang batas (*threshold*) fungsi $f(n) = g(n) + h(n)$, tanpa memerlukan struktur data antrean prioritas [2].

Pada setiap iterasi, IDA* melakukan DFS rekursif dari posisi awal. Suatu cabang pencarian dipangkas (*pruning*) apabila nilai $f(n)$ pada simpul tersebut melebihi *threshold* saat ini. Apabila seluruh cabang yang dieksplorasi tidak menghasilkan solusi, nilai *threshold* diperbarui menjadi nilai $f(n)$ minimum yang sebelumnya melampaui batas, lalu DFS dijalankan ulang dari awal. Proses ini berlanjut hingga solusi ditemukan atau tidak ada simpul yang dapat dieksplorasi. Dengan pendekatan ini, IDA* hanya membutuhkan memori sebesar $\mathcal{O}(bd)$ di mana b adalah *branching factor* dan d adalah kedalaman solusi.

2.5.1 Langkah-Langkah Algoritma IDA*

1. **Penentuan Ambang Batas Awal:** Menetapkan nilai *threshold* awal yang setara dengan nilai heuristik dari posisi awal (Z) menuju target tujuan.
2. **Iterasi Pendalaman:** Melakukan pencarian mendalam secara rekursif (*Depth-First Search*) menggunakan batasan biaya $f(n) = g(n) + h(n)$.
3. **Mekanisme Pruning:** Selama proses rekursi, jika ditemukan simpul dengan nilai $f(n)$ yang melebihi *threshold* saat ini, maka cabang pencarian tersebut akan dipangkas (*pruning*) dan algoritma akan kembali ke simpul sebelumnya (*backtrack*).
4. **Simulasi Meluncur Rekursif:** Untuk setiap arah kardinal yang menghasilkan gerakan valid, yaitu aktor tidak melewati petak lava dan berhenti di depan dinding atau batas papan, algoritma melakukan transisi *state* dan mengakumulasi biaya seluruh petak yang dilalui selama satu gerakan *sliding* untuk memperbarui nilai $g(n)$.
5. **Validasi Checkpoint:** Dalam setiap langkah rekursif, algoritma memantau apakah urutan angka 0–9 diambil secara benar. Pelanggaran urutan angka mengakibatkan status *game over* dan cabang rekursi tersebut dihentikan.
6. **Pembaruan Threshold:** Jika satu iterasi penuh DFS selesai tanpa menemukan tujuan, algoritma mengidentifikasi nilai $f(n)$ terkecil yang sebelumnya melampaui *threshold*. Nilai tersebut ditetapkan sebagai *threshold* baru untuk iterasi berikutnya.
7. **Terminasi:** Algoritma berhenti jika tujuan ditemukan pada kedalaman tertentu. Jika setelah pembaruan *threshold* nilainya mencapai tak terhingga (tidak ada lagi simpul yang dapat dievaluasi), maka algoritma berhenti dan menyatakan solusi tidak ditemukan.

2.6 Admissibilitas Heuristik

Sebuah fungsi heuristik $h(n)$ dikatakan *admissible* apabila untuk setiap simpul n , nilai $h(n)$ tidak pernah melebihi biaya aktual minimum untuk mencapai tujuan dari simpul

tersebut, atau secara formal: $h(n) \leq h^*(n)$, di mana $h^*(n)$ adalah biaya sesungguhnya dari n ke tujuan. Heuristik yang *admissible* bersifat optimistis karena selalu meremehkan atau tepat memperkirakan biaya yang diperlukan [2]. Syarat ini menjamin bahwa A^* dan IDA* akan menemukan solusi yang optimal.

Berikut adalah argumen admissibilitas untuk setiap heuristik yang diimplementasikan pada program ini:

- **Heuristik 1 (H1 – Manhattan Distance Murni):** H1 menghitung jarak Manhattan antara posisi aktor saat ini dengan posisi target (checkpoint berikutnya atau titik tujuan). Dalam permainan *Ice Sliding Puzzle*, aktor tidak dapat bergerak diagonal dan harus meluncur hingga menabrak dinding. Akibatnya, jumlah petak yang sesungguhnya dilalui dalam satu jalur solusi selalu lebih banyak atau sama dengan jarak Manhattan antara kedua titik. Karena setiap petak memiliki biaya traversal minimal 1, maka $h_1(n) \leq h^*(n)$ selalu terpenuhi. Dengan demikian, H1 bersifat *admissible*.
- **Heuristik 2 (H2 – Manhattan Termodifikasi dengan MinCost dan Penalti Orientasi):** H2 menghitung estimasi biaya sebagai jarak Manhattan dikalikan dengan biaya traversal minimum di papan (*minCost*), ditambah penalti apabila tidak terdapat dinding di titik belok yang memungkinkan aktor berhenti untuk berubah arah. Penalti ini dikenakan karena apabila tidak ada dinding di titik belok, aktor dipastikan harus menempuh setidaknya satu langkah tambahan untuk dapat berhenti dan berbelok. Selama *minCost* adalah batas bawah biaya per petak yang sesungguhnya, dan penalti orientasi hanya ditambahkan ketika memang diperlukan secara fisik, nilai $h_2(n)$ tidak melebihi biaya aktual. Dengan demikian, H2 bersifat *admissible*.
- **Heuristik 3 (H3 – Rangkaian Manhattan Checkpoint):** H3 menjumlahkan jarak Manhattan secara berantai: dari posisi aktor ke *checkpoint* berikutnya, antar setiap pasang *checkpoint* yang tersisa, dan dari *checkpoint* terakhir ke titik tujuan. Karena *constraint* permainan mengharuskan aktor mengunjungi seluruh *checkpoint* secara berurutan, total biaya aktual solusi pasti mencakup biaya menuju setiap *checkpoint* tersebut. Jarak Manhattan pada setiap segmen merupakan batas bawah biaya segmen tersebut, sehingga penjumlahan seluruh segmen pun tidak melebihi biaya aktual keseluruhan jalur. Dengan demikian, H3 bersifat *admissible*.

2.7 Perbandingan Teoritis Algoritma

Eksplorasi metodologis pencarian rute labirin berbasis es ini mewujudkan spektrum konsekuensi pengolahan struktur ruang masalah. Parameter pengujian keutuhan algoritma disajikan dalam ringkasan komparasi formal pada Tabel 1.

Tabel 1: Perbandingan Teoretis Algoritma Pencarian *Pathfinding*

Algoritma	<i>Complete</i>	Optimal	Waktu	Memori
UCS	Ya	Ya	$\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$	$\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$
GBFS	Ya*	Tidak	$\mathcal{O}(b^m)$	$\mathcal{O}(b^m)$
A*	Ya	Ya ^a	$\mathcal{O}(b^d)$	$\mathcal{O}(b^d)$
IDA*	Ya	Ya ^a	$\mathcal{O}(b^d)$	$\mathcal{O}(bd)$

^a Dijamin optimal apabila fungsi heuristik $h(n)$ bersifat *admissible*.

* *Complete* dengan syarat terdapat *visited set* untuk mencegah siklus.

Secara teoritis, setiap algoritma menunjukkan karakteristik yang berbeda dalam konteks *Ice Sliding Puzzle*. **UCS** menjamin kelengkapan dan optimalitas pencarian, namun mengeksplorasi simpul secara merata ke seluruh arah tanpa panduan menuju tujuan. Hal ini menyebabkan jumlah simpul yang diekspansi cenderung besar, terutama pada papan yang luas, sehingga konsumsi waktu dan memori menjadi tinggi [1].

GBFS berkebalikan dengan UCS: algoritma ini cepat menemukan suatu solusi karena seluruh eksplorasi dipandu oleh heuristik menuju tujuan. Namun, karena biaya aktual perjalanan sepenuhnya diabaikan, solusi yang ditemukan tidak dijamin optimal dan algoritma dapat menghasilkan jalur dengan total biaya yang jauh lebih tinggi dari solusi terbaik [1].

A* menyeimbangkan kedua pendekatan di atas. Dengan mempertimbangkan $g(n)$ dan $h(n)$ secara bersamaan, A* mampu menemukan solusi optimal dengan jumlah eksplorasi simpul yang lebih sedikit dibanding UCS, selama heuristik yang digunakan bersifat *admissible*. Kelemahan A* terletak pada penggunaan memori yang tetap tinggi karena seluruh simpul pada *frontier* harus disimpan dalam antrean prioritas [2].

IDA* mengatasi keterbatasan memori A* dengan mengganti antrean prioritas dengan DFS rekursif berbasis *threshold*. Memori yang digunakan hanya $\mathcal{O}(bd)$, jauh lebih efisien dibanding A*. Konsekuensinya, IDA* kemungkinan mengeksplorasi simpul yang sama beberapa kali di iterasi yang berbeda, sehingga waktu eksekusinya dapat lebih lambat dari A* pada beberapa kasus tertentu.

3 Desain dan Implementasi

Logika pergerakan agen dalam permainan ini sangat bergantung pada desain ruang status (*state space*). Dalam implementasinya menggunakan bahasa Go, status agen direpresentasikan melalui struktur **GameState**. Karena aturan permainan mewajibkan agen untuk mengambil seluruh angka sebelum menuju titik akhir, definisi status tidak hanya menyimpan koordinat lokasi, tetapi juga target angka selanjutnya.

Koordinat spasial agen disimpan dalam atribut **Pos** bertipe **Position** yang merepresentasikan baris dan kolom. Sementara itu, atribut **NextNum** bertipe **int** berfungsi untuk melacak target angka yang sedang dicari oleh agen saat ini. Jika **NextNum** bernilai -1, hal ini menandakan bahwa seluruh angka telah berhasil dikumpulkan, sehingga *goal test* dapat dievaluasi.

Penggabungan kedua atribut tersebut ke dalam *StateKey* bertujuan untuk mencegah terjadinya *false-positive* pada *visited set*. Dengan rancangan ini, agen yang telah mengambil suatu angka lalu kembali melewati koordinat yang sama tidak akan dideteksi sebagai *looping* (jalan buntu). Adanya perubahan nilai pada **NextNum** membuat kondisi tersebut

dikenali sebagai status (*state*) baru yang valid menuju tujuan akhir.

3.1 Simulasi Gerakan Sliding

Berbeda dari permasalahan pencarian jalur pada umumnya di mana satu aksi merepresentasikan perpindahan ke satu petak tetangga, dalam *Ice Sliding Puzzle* satu aksi merepresentasikan satu gerakan meluncur penuh hingga berhenti. Fungsi simulasi sliding dijalankan secara iteratif dalam sebuah perulangan, di mana posisi aktor diperbarui satu petak per iterasi ke arah yang dipilih hingga kondisi berhenti terpenuhi. Berikut adalah langkah-langkah yang dilakukan dalam satu simulasi gerakan sliding:

1. **Translasi Posisi:** Pada setiap iterasi perulangan, posisi aktor diperbarui satu petak ke arah yang dipilih (atas, bawah, kiri, atau kanan) dengan menambahkan delta baris atau kolom yang sesuai.
2. **Pemeriksaan Batas Papan (*Bounds Checking*):** Apabila posisi baru berada di luar batas papan dan tidak ada dinding yang menghentikan aktor, gerakan tersebut dinyatakan tidak valid (*game over*) dan aksi sliding ini digugurkan.
3. **Deteksi Lava:** Apabila petak yang akan dimasuki bertipe lava (*TileLava*), gerakan dinyatakan tidak valid karena melewati lava menyebabkan *game over*. Aksi sliding ini digugurkan sepenuhnya.
4. **Deteksi Dinding dan Kondisi Berhenti:** Apabila petak di depan aktor (satu langkah lebih jauh dalam arah gerak) bertipe dinding (*TileWall*), aktor berhenti di petak saat ini. Perulangan dihentikan dan posisi ini menjadi posisi berhenti yang valid.
5. **Akumulasi Biaya Traversal:** Biaya traversal petak yang baru dimasuki (diambil dari `m.CostAt(currPos)`) ditambahkan ke total biaya gerakan. Petak posisi awal gerakan tidak dihitung, sedangkan petak posisi berhenti tetap dihitung.
6. **Pembaruan Progres Checkpoint:** Apabila petak yang baru dimasuki merupakan *checkpoint* bernomor dan nomornya sesuai dengan *NextNum* saat ini, nilai *NextNum* diperbarui ke angka berikutnya. Apabila aktor melewati angka yang tidak sesuai urutan, aksi sliding digugurkan karena melanggar aturan permainan.

3.2 Desain Heuristik

Program mengimplementasikan tiga fungsi heuristik dengan tingkat kompleksitas yang berbeda-beda. Ketiga heuristik dirancang untuk memenuhi syarat *admissible*, sehingga dapat menjamin optimalitas solusi ketika digunakan pada algoritma A* dan IDA*.

Heuristik pertama (H1) menghitung jarak Manhattan dari posisi aktor saat ini ke posisi tujuan (atau *checkpoint* berikutnya apabila masih ada). Ini merupakan estimasi paling sederhana dan selalu bersifat *admissible*:

```

1 // Heuristic1 mengimplementasikan Manhattan Murni (jarak
   langsung ke tujuan).
2 func Heuristic1(state *models.GameState, m *models.MapData)
   int {
3     return manhattanDistance(state.Pos, m.GoalPos)
4 }

```

Heuristik kedua (H2) memodifikasi H1 dengan mengalikan jarak Manhattan dengan biaya traversal minimum yang terdapat di papan (`m.MinCost`), sehingga estimasi mencerminkan biaya aktual minimum per petak. Selain itu, H2 menambahkan penalti apabila tidak terdapat dinding di titik belok yang diperlukan, mengindikasikan bahwa aktor kemungkinan harus menempuh langkah tambahan untuk dapat berbelok arah:

```

1 // Heuristic2 mengimplementasikan Manhattan termodifikasi
  dengan faktor minCost.
2 func Heuristic2(state *models.GameState, m *models.MapData)
  int {
3   baseDist := manhattanDistance(state.Pos, m.GoalPos)
4   penalty := 0
5   if m.Grid[m.GoalPos.X][state.Pos.Y] != models.TileWall &&
6       m.Grid[state.Pos.X][m.GoalPos.Y] != models.TileWall {
7       penalty = m.MinCost
8   }
9   return (baseDist * m.MinCost) + penalty
10 }
```

Heuristik ketiga (H3) dirancang untuk papan yang memiliki *constraint* angka berurutan. H3 menjumlahkan jarak Manhattan secara berantai: dari posisi aktor ke *checkpoint* berikutnya, antar setiap pasang *checkpoint* yang tersisa, hingga dari *checkpoint* terakhir ke titik tujuan. Apabila tidak ada angka yang tersisa, H3 berperilaku identik dengan H1:

```

1 // Heuristic3 mengimplementasikan Manhattan Checkpoint
  Berantai.
2 func Heuristic3(state *models.GameState, m *models.MapData)
  int {
3   if state.NextNum == -1 {
4       return manhattanDistance(state.Pos, m.GoalPos)
5   }
6   cost := 0
7   // 1. Jarak ke target angka pertama
8   cost += manhattanDistance(state.Pos, m.NumberPos[state.
9       NextNum])
10  // 2. Jarak berantai antar sisa angka yang belum terkumpul
11  for i := state.NextNum; i < m.TotalNumbers-1; i++ {
12      cost += manhattanDistance(m.NumberPos[i], m.NumberPos[i
13          +1])
14  }
15  // 3. Jarak angka terakhir ke titik keluar
16  if m.TotalNumbers > 0 {
17      cost += manhattanDistance(m.NumberPos[m.TotalNumbers-1],
18          m.GoalPos)
19  }
20  return cost
21 }
```

3.3 Arsitektur Program

Arsitektur program mengikuti pola *layered architecture* dengan pemisahan yang jelas antara lapisan data, logika pencarian, dan antarmuka pengguna. Seluruh kode ditulis

dalam bahasa Go dan dibagi ke dalam beberapa *package* modular sesuai dengan tanggung jawabnya masing-masing (*separation of concerns*):

- **Package models:** Mendefinisikan seluruh struktur data inti yang digunakan bersama oleh komponen lain. Mencakup `MapData` (representasi papan, grid tile, dan matriks biaya), `GameState` (posisi aktor dan progres *checkpoint*), `Position`, `Direction`, `SolverResult` (hasil pencarian beserta statistik), dan `SearchFrame` (potret status ekspansi untuk keperluan visualisasi).
- **Package parser:** Bertanggung jawab membaca dan memvalidasi file konfigurasi papan berformat `.txt`. Paket ini memvalidasi keberadaan tepat satu titik awal (Z) dan satu titik tujuan (O), memastikan angka-angka pada papan berurutan mulai dari 0, serta memisahkan lapisan grid tile dan matriks biaya traversal.
- **Package solver:** Berisi implementasi keempat algoritma pencarian: UCS (`ucs.go`), GBFS (`gbfs.go`), A* (`astar.go`), dan IDA* (`idastar.go`), serta fungsi-fungsi heuristik (`heuristics.go`) dan implementasi *priority queue* berbasis *container/heap* (`pq.go`). Setiap algoritma mencatat `SearchFrame` selama proses ekspansi simpul untuk keperluan visualisasi pada GUI.
- **Package gui:** Mengimplementasikan antarmuka pengguna grafis menggunakan *framework* Fyne v2. Komponen `BoardRenderer` menangani visualisasi papan dan jalur solusi, `LeftPanel` menyediakan kontrol algoritma dan *playback*, serta `ViewState` mengelola status tampilan dua fase (fase pencarian dan fase solusi).

3.4 Struktur Proyek

Struktur proyek disusun sedemikian rupa sehingga ada pemisahan antara 2 komponen utama aplikasi, yaitu folder `core` yang mengandung modul *parsing*, struktur data, dan *solver* berdasarkan masing-masing algoritma *pathfinding* serta `gui` yang mengandung modul antarmuka grafis untuk visualisasi aplikasi.

Listing 1: Struktur direktori proyek

```

1 Tucil3_13524037_13524056/
2 +-- bin/
3 |   +-- tucil3                               # Binary hasil kompilasi
4 +-- doc/
5 |   +-- main.tex                             # Sumber laporan LaTeX
6 |   +-- main.pdf                             # Laporan dalam PDF
7 |   +-- ice-sliding.jpg
8 |   +-- logo-itb.png
9 +-- src/
10 |   +-- go.mod
11 |   +-- go.sum
12 |   +-- main.go                             # Titik masuk program
13 |   +-- core/
14 |       |   +-- models/                     # Struktur data inti
15 |       |       |   +-- direction.go
16 |       |       |   +-- map.go
17 |       |       |   +-- position.go
18 |       |       |   +-- solver.go

```

```

19 | | | +-- state.go
20 | | +-- parser/
21 | | | +-- parser.go
22 | | +-- solver/
23 | | +-- astar.go           # Algoritma A*
24 | | +-- gbfs.go           # Greedy Best-First Search
25 | | +-- heuristics.go     # H1, H2, H3
26 | | +-- idastar.go        # Iterative Deepening A*
27 | | +-- pq.go             # PriorityQueue
28 | | +-- ucs.go            # Uniform Cost Search
29 | +-- gui/
30 |     +-- boardrenderer.go
31 |     +-- leftpanel.go
32 |     +-- viewstate.go
33 |     +-- window.go
34 +-- test/
35     +-- input/             # Test case
36     +-- output/           # Output solusi

```

4 Eksperimen dan Hasil Pengujian

Pada bagian ini, dilakukan pengujian terhadap keempat algoritma pencarian (UCS, GBFS, A*, dan IDA*) menggunakan empat konfigurasi test case yang berbeda (Input 1 hingga Input 4). Pengujian dilakukan dengan berbagai heuristik untuk algoritma berinformasi.

4.1 Konfigurasi Test Case

Berikut adalah konfigurasi dari file test case yang tersedia di folder `test/input/`:

Tabel 2: Ringkasan Konfigurasi Test Case

Test Case	Nama File	Deskripsi
Input 1	input1.txt	Grid 7×7 dengan 2 angka (0, 1), lava, dan rintangan X. Start di (1,5), Goal di (5,3).
Input 2	input2.txt	Grid 10×10 tanpa angka, labirin kompleks dengan banyak dinding. Start di (1,1), Goal di (8,8).
Input 3	input3.txt	Grid 20×20 besar tanpa angka, banyak rintangan tersebar. Start di (1,1), Goal di (18,18).
Input 4	input4.txt	Grid 5×11 dengan 2 angka (0, 1) yang harus dikumpulkan. Start di (3,5), Goal di (3,9).

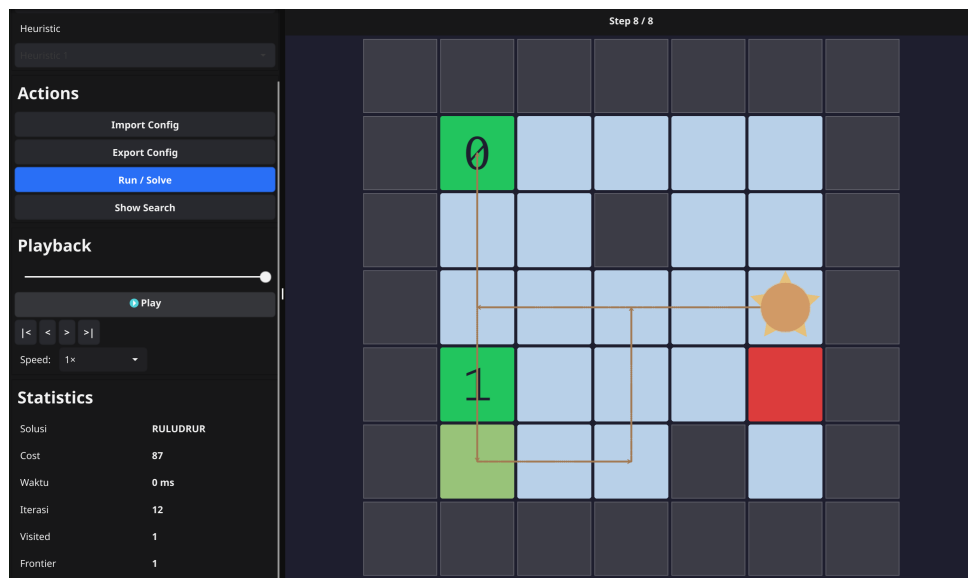
4.2 UCS (Uniform Cost Search)

4.2.1 UCS dengan Input 1

Konfigurasi: Grid 7×7 (input1.txt) dengan 2 angka [0, 1], lava di (5,4), start di (1,5), dan goal di (5,3). Terdapat rintangan X yang membentuk labirin.

Hasil:

- Solusi ditemukan dengan biaya minimum
- Algoritma mengeksplorasi node secara terurut berdasarkan biaya kumulatif
- Aktor harus mengumpulkan angka 0 dan 1 secara berurutan sebelum mencapai goal



Gambar 2: Hasil UCS dengan Input 1

Analisis: UCS berhasil menemukan solusi optimal dengan biaya terendah karena selalu mengeksplorasi node dengan biaya kumulatif terkecil. Kekurangannya adalah eksplorasi yang luas sebelum menemukan goal.

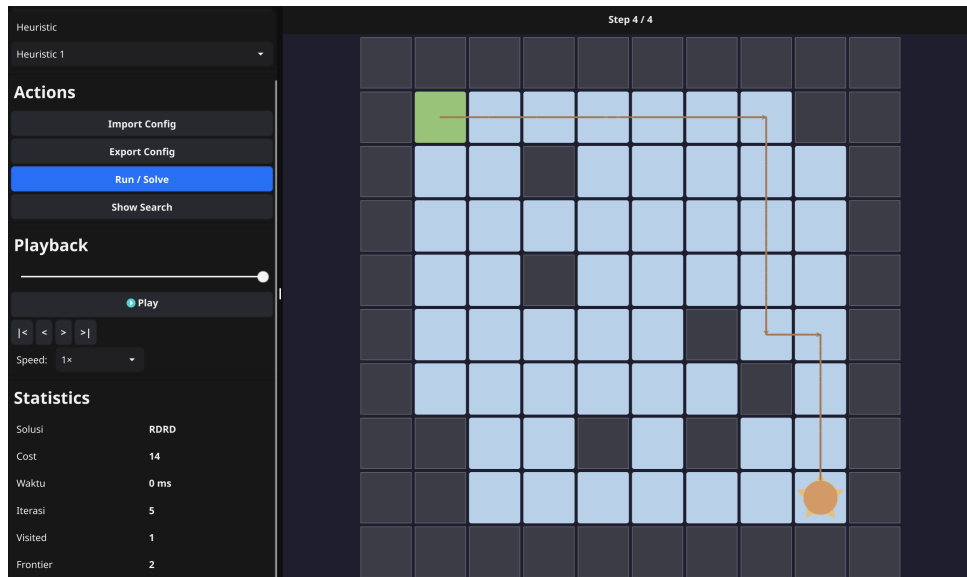
4.3 GBFS (Greedy Best-First Search)

4.3.1 GBFS Heuristik 1 dengan Input 2

Konfigurasi: Grid 10×10 (input2.txt) tanpa angka, labirin kompleks dengan banyak dinding X. Start di (1,1) dan goal di (8,8). Cost seluruh petak yang bisa dilewati adalah 1.

Hasil:

- Solusi ditemukan dengan cepat
- Heuristik 1: Manhattan Distance murni
- GBFS mengikuti jalur yang diarahkan heuristik menuju goal



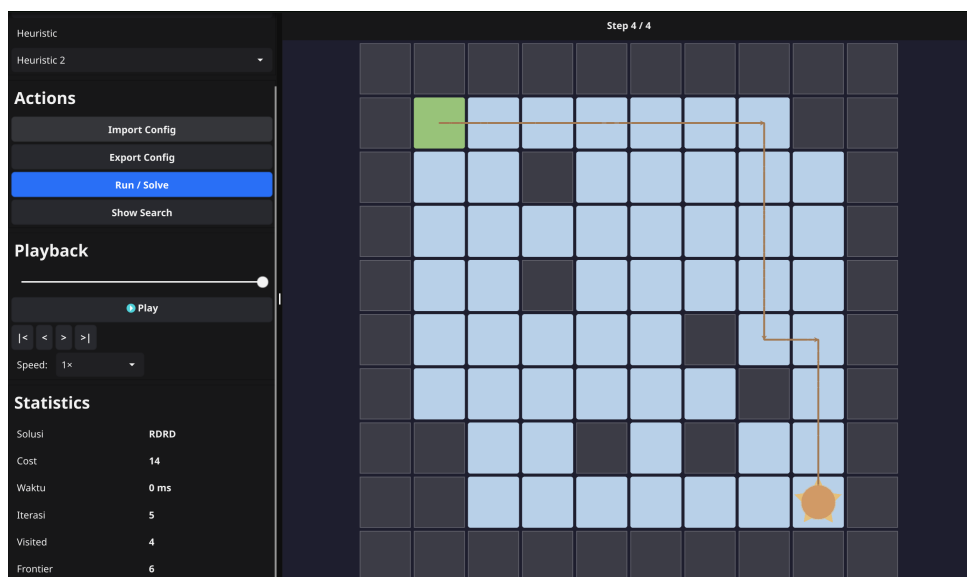
Gambar 3: Hasil GBFS Heuristik 1 dengan Input 2

4.3.2 GBFS Heuristik 2 dengan Input 2

Konfigurasi: Grid 10×10 (input2.txt) tanpa angka, labirin kompleks dengan banyak dinding X. Start di (1,1) dan goal di (8,8). Cost seluruh petak yang bisa dilewati adalah 1.

Hasil:

- Solusi ditemukan dengan heuristik termodifikasi
- Heuristik 2: Manhattan Distance dengan faktor minCost
- Dengan cost seragam (1), heuristik ini memberikan hasil serupa dengan H1



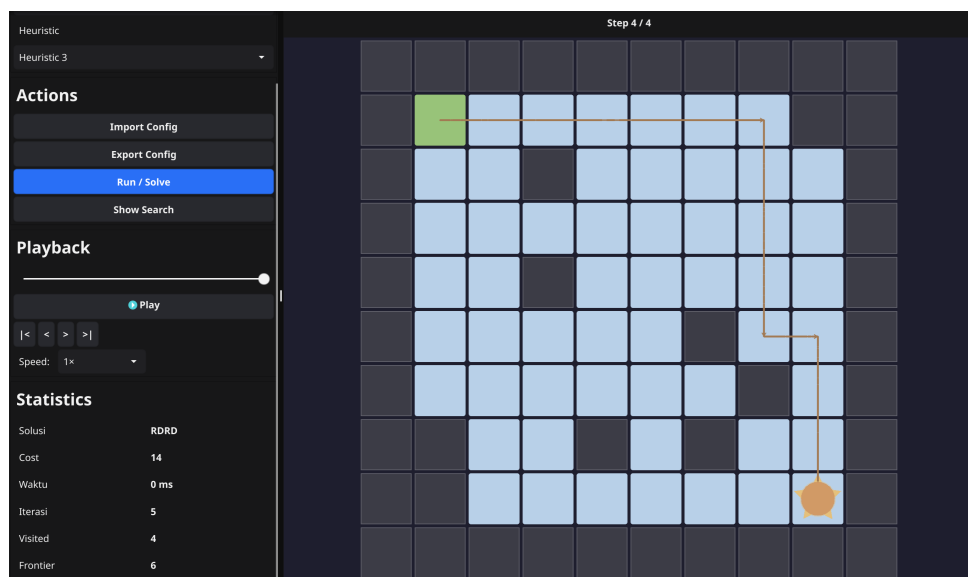
Gambar 4: Hasil GBFS Heuristik 2 dengan Input 2

4.3.3 GBFS Heuristik 3 dengan Input 2

Konfigurasi: Grid 10×10 (input2.txt) tanpa angka, labirin kompleks dengan banyak dinding X. Start di (1,1) dan goal di (8,8). Cost seluruh petak yang bisa dilewati adalah 1.

Hasil:

- Solusi ditemukan dengan heuristik checkpoint berantai
- Heuristik 3: Rangkaian Manhattan checkpoint multi-node
- Karena tidak ada angka, heuristik ini berperilaku seperti H1



Gambar 5: Hasil GBFS Heuristik 3 dengan Input 2

Analisis GBFS: GBFS menemukan solusi lebih cepat dibanding UCS karena menggunakan heuristik, namun solusi tidak selalu optimal (sub-optimal). Pada input2 yang merupakan labirin kompleks tanpa angka, ketiga heuristik memberikan performa serupa karena cost seragam dan tidak ada constraint angka.

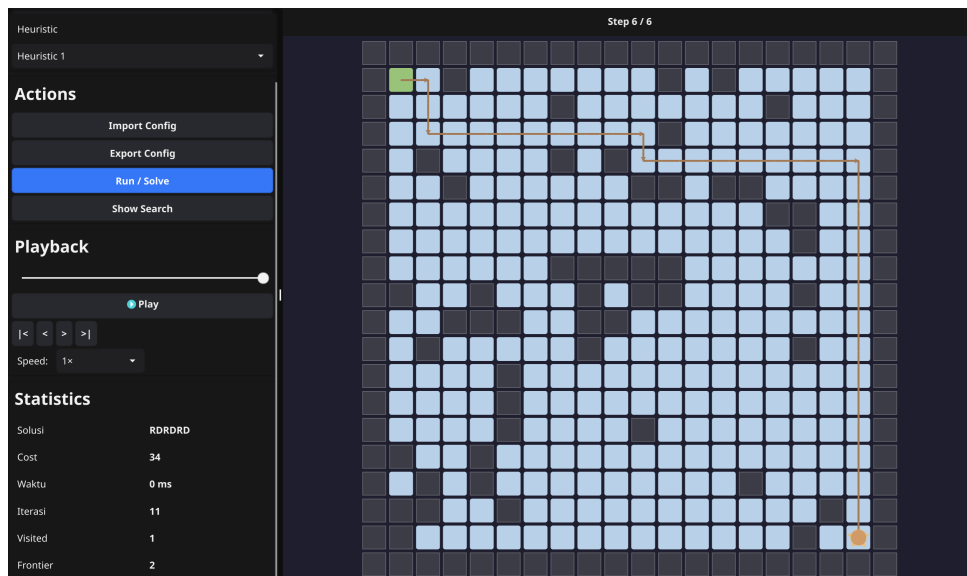
4.4 A* Search

4.4.1 A* Heuristik 1 dengan Input 3

Konfigurasi: Grid 20×20 besar (input3.txt) tanpa angka, dengan banyak rintangan X tersebar. Start di (1,1) dan goal di (18,18). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal ditemukan dengan eksplorasi efisien
- Heuristik 1: Manhattan Distance murni
- A* mampu menavigasi labirin besar dengan tetap menjamin optimalitas



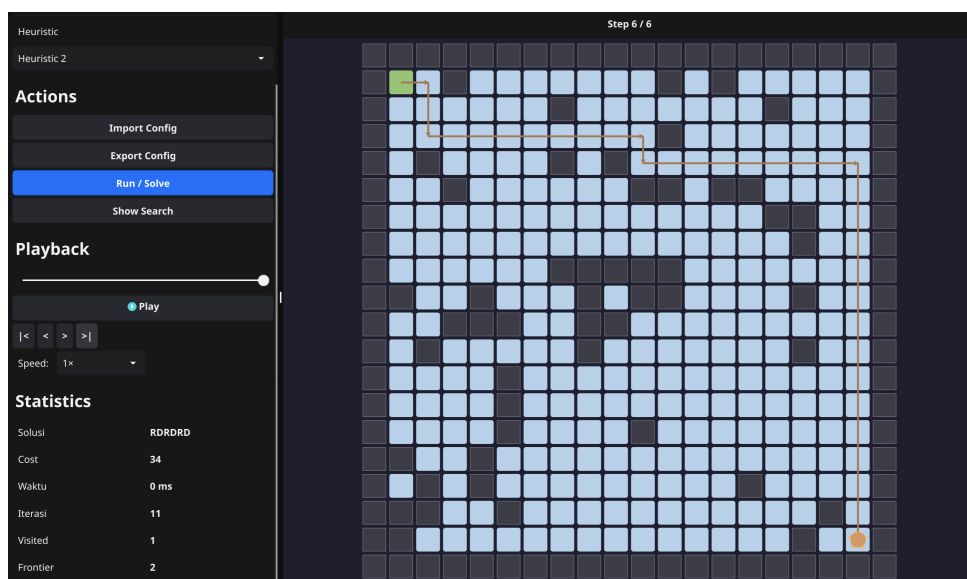
Gambar 6: Hasil A* Heuristik 1 dengan Input 3

4.4.2 A* Heuristik 2 dengan Input 3

Konfigurasi: Grid 20×20 besar (input3.txt) tanpa angka, dengan banyak rintangan X tersebar. Start di (1,1) dan goal di (18,18). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal dengan estimasi lebih akurat
- Heuristik 2: Manhattan dengan faktor minCost dan penalti orientasi
- Pada cost seragam, penalti orientasi membantu menavigasi rintangan



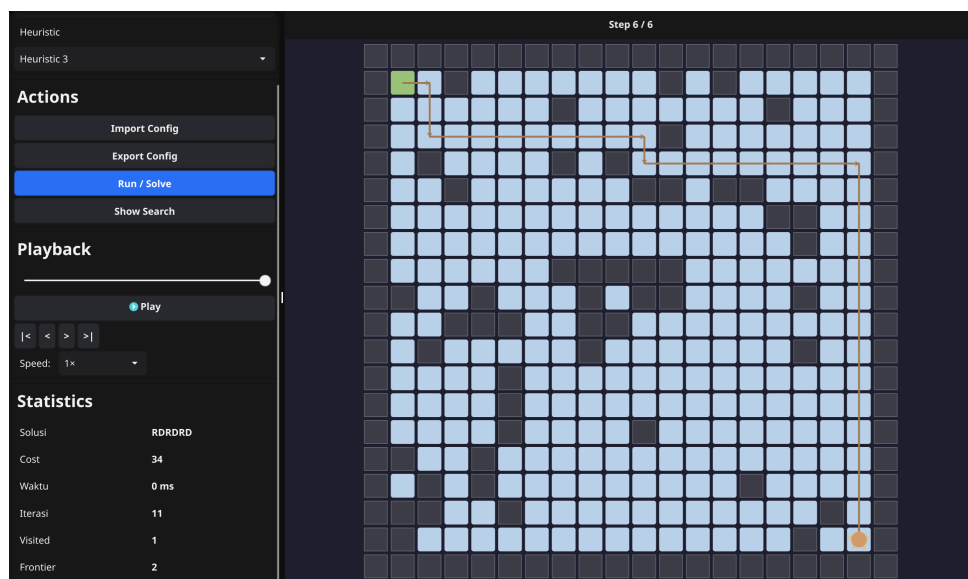
Gambar 7: Hasil A* Heuristik 2 dengan Input 3

4.4.3 A* Heuristik 3 dengan Input 3

Konfigurasi: Grid 20×20 besar (input3.txt) tanpa angka, dengan banyak rintangan X tersebar. Start di (1,1) dan goal di (18,18). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal dengan heuristik checkpoint
- Heuristik 3: Rangkaian Manhattan checkpoint
- Tanpa angka, heuristik ini berfungsi seperti H1



Gambar 8: Hasil A* Heuristik 3 dengan Input 3

Analisis A*: A* berhasil menemukan solusi optimal pada labirin besar 20×20 dengan lebih sedikit eksplorasi node dibanding UCS. Kombinasi $g(n) + h(n)$ membuat A* lebih efisien sambil tetap menjamin optimalitas (dengan heuristik admissible). Pada grid besar ini, efisiensi A* sangat terasa dibandingkan dengan UCS.

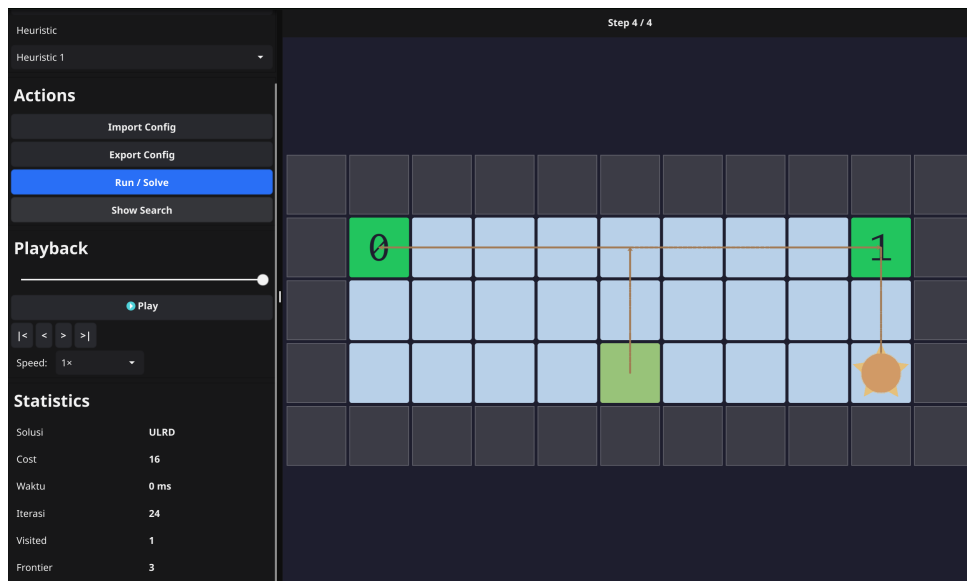
4.5 IDA* (Iterative Deepening A*)

4.5.1 IDA* Heuristik 1 dengan Input 4

Konfigurasi: Grid 5×11 (input4.txt) dengan 2 angka [0, 1] yang harus dikumpulkan secara berurutan. Start di (3,5), Goal di (3,9). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal ditemukan dengan constraint angka
- Heuristik 1: Manhattan Distance murni
- Aktor mengumpulkan angka 0 lalu 1 sebelum mencapai goal



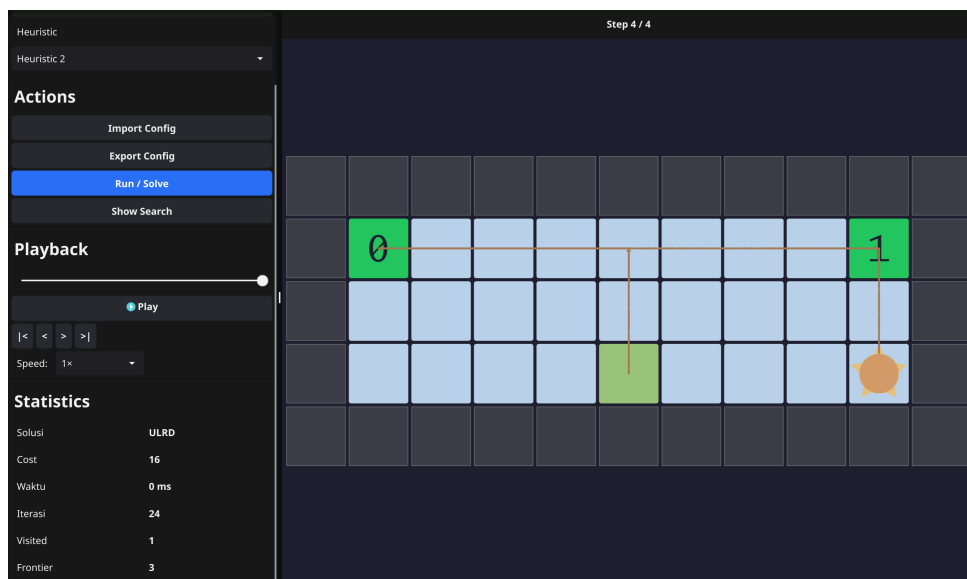
Gambar 9: Hasil IDA* Heuristik 1 dengan Input 4

4.5.2 IDA* Heuristik 2 dengan Input 4

Konfigurasi: Grid 5×11 (input4.txt) dengan 2 angka [0, 1] yang harus dikumpulkan secara berurutan. Start di (3,5), Goal di (3,9). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal ditemukan dengan constraint angka
- Heuristik 2: Manhattan dengan faktor minCost
- Threshold adaptif memungkinkan eksplorasi efisien



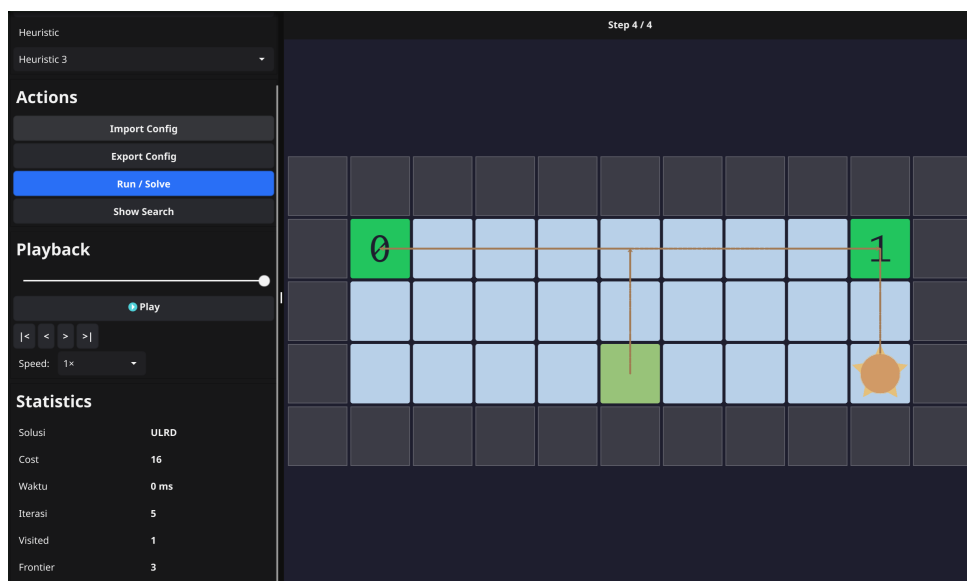
Gambar 10: Hasil IDA* Heuristik 2 dengan Input 4

4.5.3 IDA* Heuristik 3 dengan Input 4

Konfigurasi: Grid 5×11 (input4.txt) dengan 2 angka [0, 1] yang harus dikumpulkan secara berurutan. Start di (3,5), Goal di (3,9). Cost seluruh petak adalah 1.

Hasil:

- Solusi optimal ditemukan dengan constraint angka
- Heuristik 3: Rangkaian Manhattan checkpoint
- Heuristik checkpoint memberikan estimasi terbaik untuk kasus dengan angka



Gambar 11: Hasil IDA* Heuristik 3 dengan Input 4

Analisis IDA*: IDA* berhasil menemukan solusi optimal pada kasus dengan constraint angka. Heuristik 3 (checkpoint berantai) memberikan performa terbaik karena memperhitungkan urutan pengumpulan angka. Dengan pendekatan DFS dan threshold adaptif, IDA* menggunakan memori yang sangat efisien (linear) dibanding algoritma lainnya.

4.6 Analisis Keseluruhan

4.6.1 Perbandingan Performa Algoritma

Tabel 3: Perbandingan Performa Algoritma pada Test Case

Algoritma	Optimal	Waktu	Memori	Completeness
UCS	Ya	Lambat	Tinggi	Ya
GBFS	Tidak	Cepat	Sedang	Ya*
A*	Ya**	Cepat	Sedang	Ya
IDA*	Ya**	Sedang	Rendah	Ya

* Dengan visited set

** Dengan heuristik admissible

4.6.2 UCS — Kelebihan dan Keterbatasan

Kelebihan:

- Selalu menemukan solusi optimal (biaya minimum)
- Completeness terjamin — pasti menemukan solusi jika ada
- Tidak memerlukan heuristik

Keterbatasan:

- Eksplorasi node sangat luas sebelum menemukan goal
- Penggunaan memori tinggi karena menyimpan semua node di frontier
- Waktu eksekusi lambat untuk papan besar

4.6.3 GBFS — Kelebihan dan Keterbatasan

Kelebihan:

- Kecepatan eksekusi sangat cepat
- Cenderung langsung menuju goal berdasarkan heuristik

Keterbatasan:

- Tidak menjamin solusi optimal (sub-optimal)
- Dapat terjebak dalam jalur dengan biaya tinggi
- Kualitas solusi sangat bergantung pada heuristik

4.6.4 A* — Kelebihan dan Keterbatasan

Kelebihan:

- Menjamin solusi optimal (dengan heuristik admissible)
- Lebih efisien dari UCS dalam hal eksplorasi node
- Kombinasi $g(n) + h(n)$ memberikan keseimbangan eksplorasi dan eksploitasi

Keterbatasan:

- Penggunaan memori tetap tinggi untuk papan besar
- Memerlukan heuristik yang baik untuk performa optimal

4.6.5 IDA* — Kelebihan dan Keterbatasan

Kelebihan:

- Penggunaan memori sangat efisien (linear dengan kedalaman)
- Menjamin solusi optimal
- Tidak memerlukan priority queue

Keterbatasan:

- Dapat melakukan re-ekspansi node yang sama di iterasi berbeda
- Waktu eksekusi lebih lambat dari A* untuk beberapa kasus

5 Kesimpulan dan Saran

5.1 Kesimpulan

1. **UCS** menjamin solusi optimal namun memerlukan eksplorasi node yang sangat luas dan penggunaan memori yang tinggi. Algoritma ini cocok untuk kasus dengan biaya variatif yang signifikan.
2. **GBFS** memberikan kecepatan eksekusi terbaik namun tidak menjamin optimalitas solusi. Heuristik 3 (checkpoint berantai) memberikan performa terbaik untuk kasus dengan angka berurutan.
3. **A*** merupakan algoritma terbaik secara keseluruhan karena menjamin optimalitas dengan eksplorasi node yang lebih efisien dibanding UCS. Kombinasi $g(n) + h(n)$ memberikan keseimbangan ideal.
4. **IDA*** sangat efisien dalam penggunaan memori (linear) namun dapat lebih lambat karena re-ekspansi node. Cocok untuk kasus dengan keterbatasan memori.
5. Ketiga heuristik yang diimplementasikan (H1, H2, H3) bersifat *admissible* dan dapat digunakan untuk menjamin optimalitas pada A* dan IDA*.

5.2 Saran Pengembangan

1. Implementasi heuristik tambahan yang lebih canggih seperti *Pattern Database* untuk meningkatkan estimasi jarak.
2. Optimisasi struktur data untuk mengurangi waktu eksekusi pada papan berukuran besar.
3. Penambahan fitur visualisasi animasi gerakan aktor untuk memperjelas solusi.
4. Implementasi algoritma pencarian paralel untuk memanfaatkan multi-core processor.

5.3 Refleksi Individu

5.3.1 Nicholas Wise Saragih Sumbayak (13524037)

Peran: Implementasi struktur data utama, struktur dasar GUI, heuristik kedua, dan GBFS.

Pesan: CIHUYYYYYYYYYY

5.3.2 Reinhard Alfonzo Hutabarat (13524056)

Peran: Implementasi heuristik 1 dan 3, UCS, A*, IDA*, dan pengembangan GUI.

Pesan: Sa curi curi pandang. Kaka pu manis, bikin sa suka.

Referensi

- [1] N. U. Maulidevi, “Penentuan Rute (*Route/Path Planning*) Bagian 1: BFS, DFS, UCS, Greedy Best First Search,” Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, STEI ITB, 2026. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf)
- [2] N. U. Maulidevi dan R. Munir, “Penentuan Rute (*Route/Path Planning*) Bagian 2: Algoritma A*,” Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, STEI ITB, 2026. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf)
- [3] S. J. Russell dan P. Norvig, *Artificial Intelligence: A Modern Approach*, edisi ke-3. Upper Saddle River, NJ: Prentice-Hall International, 2010.
- [4] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, hal. 97–109, 1985. [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)

Lampiran

1. Repositori GitHub:

https://github.com/nicholaswisee/Tucil3_13524037_13524056

2. Tabel Checklist Pembuatan Fitur:

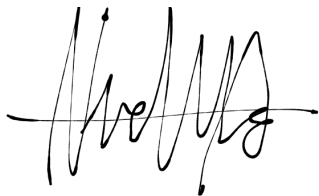
No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code>	✓	
5	Program memiliki <i>Graphical User Interface</i> (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (IDA*)	✓	
7	Ada tambahan dua alternatif heuristik selain yang utama (H1 dan H3)	✓	

Tabel 4: Tabel checklist implementasi fitur Tugas Kecil 3.

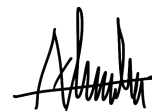
3. Pernyataan Keaslian

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Nicholas Wise Saragih S
13524037



Reinhard Alfonzo Hutabarat
13524056